

RTAI Assignment3

I. A description of your model, dataset, and verification query.

MNIST데이터셋을 사용했고 Model은 단순히 Multi-Layer Perceptron구조로 만들었다. (실제로 git의 train.py를 보면 simpleMLP이다.)

```
class SimpleMLP(nn.Module):
    def __init__(self):
        super(SimpleMLP, self).__init__()
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(28 * 28, 64)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(64, 10)

    def forward(self, x):
        x = self.flatten(x)
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        return x
```

구조는 위와 같다.

verification query는 L_∞ -norm 기준으로 모든 입력 pixel에 $\epsilon \leq 0.1$ 의 Perturbation을 적용했을 때, target class(no. 3)보다 특정 attack class(no. 8)의 예측 로짓(logit) 값이 더 높아지는 Adversarial Example가 존재하는지 탐색하도록 구성했다.

II. The results and their interpretation

Marabou SMT solver를 이용해 verification query를 실행한 결과 'SAT'판정을 받았다. 적대적 예제가 존재함을 수학적으로 증명했다.

```
=====
최종 검증 결과: SAT
=====

[1] 반례가 존재합니다 (SAT). 모델이 숫자 8로 잘못 예측하는 취약점을 찾았습니다.
구체적인 값은 다음과 같습니다:

--- 입력 변수 (Input Variables) 샘플 ---
Input[0] (변수번호 0) = -0.100000
Input[1] (변수번호 1) = -0.100000
Input[2] (변수번호 2) = -0.100000
Input[3] (변수번호 3) = -0.100000
Input[4] (변수번호 4) = -0.100000
... (나머지 생략) ...

--- 출력 변수 (Output Variables) ---
Output Class 0 (변수번호 912) = 0.004662
Output Class 1 (변수번호 913) = -0.063004
Output Class 2 (변수번호 914) = -0.016540
Output Class 3 (변수번호 915) = -0.090722
Output Class 4 (변수번호 916) = 0.038085
Output Class 5 (변수번호 917) = 0.005236
Output Class 6 (변수번호 918) = 0.062522
Output Class 7 (변수번호 919) = 0.012144
Output Class 8 (변수번호 920) = -0.090722
Output Class 9 (변수번호 921) = 0.019819
```

본 검증 과정의 총 소요 시간은 211ms이다.

```
13:04:03 Statistics update:
--- Time Statistics ---
Total time elapsed: 211 mlll (00:00:00)
Main loop: 56 mlll (00:00:00)
Preprocessing time: 142 mlll (00:00:00)
Unknown: 12 mlll (00:00:00)
Breakdown for main loop:
[50.08%] Simplex steps: 28 mlll
[42.56%] Explicit-basis bound tightening: 24 mlll
[0.00%] Constraint-matrix bound tightening: 0 mlll
[0.00%] Degradation checking: 0 mlll
[0.00%] Precision restoration: 0 mlll
[0.12%] Statistics handling: 0 mlll
[0.00%] Constraint-fixing steps: 0 mlll
[0.03%] Valid case splits: 0 mlll. Average per split: 0.00 mlll
[0.02%] Applying stored bound-tightening: 0 mlll
[0.00%] Search Tree Handler : 0 mlll
[35.65%] Symbolic Bound Tightening: 20 mlll
[12.63%] Sol-based local search: 7 mlll
[0.00%] Sol-based local search: 0 mlll
[32386004097174343.00%] Unaccounted for: 0 mlll
```

검증 엔진이 탐색해 낸 구체적인 반례(Counterexample)의 출력(Output) 값을 분석한 결과, 정답 클래스인 Output[3]의 값은 -0.090722 , 유도하려는 오답 클래스인 Output[8]의 값 역시 -0.090722 로 산출되었다.

이는 solver에 입력한 제약 조건인 $y_8 \geq y_3$ 를 완벽하게 충족하는 결과로, 미세한 pixel 조작(± 0.1)만으로도 모델의 Decision Boundary가 교란되어 모델이 숫자 3과 8을 구분하지 못하게 만들 수 있음을 보인다.

III. A brief discussion of Marabou's strengths and limitations based on your experience
(e.g., scalability, ease of use, supported model formats).

Strengths : Marabou는 SMT 기반 알고리즘을 통해 단순 시뮬레이션이 아닌 수학적 엄밀성을 바탕으로 신경망의 취약점을 증명한다. PyTorch 등으로 학습된 모델을 ONNX로 쉽게 불러와 복잡한 네트워크를 방정식화하는 편의성을 제공한다. 또한 Running Time이 0.2 sec밖에 되지 않는다.

Limitations : C++기반이라 Linux가 강제되는 아쉬움이 있었다. 또한 Scalability에 대한 한계가 있는데, parameter의 수가 많을수록 State Explosion의 위험성이 있다.